

EventsCar / GP-DEV-SINISTRO

Documentacao Tecnica Completa

Gerado automaticamente a partir dos arquivos em docs/.

Producao: <https://eventos.escsistemas.com>

Repositorio: devgpesc/GP-DEV-SINISTRO

PROMPT SISTEMA COMPLETO (Cursor IDE)

PROMPT CURSOR - EventsCar / GP-DEV-SINISTRO (Sistema Completo)

> Como usar: Copie todo este documento para uma nova conversa no Cursor (modo Agent) e peça: "Implemente/mantenha este sistema conforme a especificação abaixo."

>

> Produto: <https://eventos.escsistemas.com>

> Repositório: devgpsc/GP-DEV-SINISTRO

> Supabase Project ID: yxawavenbognqiihaesh

1. Visão do produto

Sistema SaaS multi-tenant para gestão de sinistros automotivos (associados/proteção veicular), cobrindo o ciclo:

Sinistro -> Cotação (RFQ) -> Matriz comparativa -> OC (Compra) -> Entrega -> Conclusão

Personas:

- Operador: registra sinistros, cotações, acompanha entregas
- Gerente: aprova OCs por escrito, relatórios, equipe
- Admin empresa: configurações, API, auditoria
- Super Admin plataforma: tenants, planos SaaS

2. Stack tecnológica

```
Camada | Tecnologia |
-----|-----|
Frontend | React 18 + TypeScript + Vite 5 |
Estilo | Tailwind CSS 3 |
Ícones | lucide-react |
Roteamento | react-router-dom v6 |
Backend dados | Supabase (PostgreSQL + Auth + Storage + Realtime) |
API REST | Vercel Serverless (/api/v1/) |
Auxiliar | /api/cnpj, /api/vehicles, /api/llm |
Deploy | GitHub Actions -> Vercel + Supabase migrations |
Planilhas | xlsx (export matriz) |
```

Variáveis de ambiente

Frontend (.env):

```
env
VITE_SUPABASE_URL=https://xxx.supabase.co
VITE_SUPABASE_ANON_KEY=eyJ...
VITE_API_BASE_URL=/api
VITE_APP_URL=https://eventos.escsistemas.com
```

Vercel (API serverless):

```
env
SUPABASE_URL=https://xxx.supabase.co
SUPABASE_SERVICE_ROLE_KEY=eyJ...
GEMINI_API_KEY=...
APIBRASIL_TOKEN=...
```

3. Arquitetura

```
mermaid
    Browser[React SPA]
    end
    SPA[Static dist]
    APIv1[/api/v1 REST]
    APIaux[/api/cnpj vehicles llm]
    end
    Auth[Auth PKCE]
    PG[(PostgreSQL RLS)]
    Storage[Storage buckets]
    RT[Realtime]
    end
    Browser --> SPA
    Browser --> Auth
    Browser --> PG
    Browser --> RT
    APIv1 --> PG
    Browser --> APIaux
```

Multi-tenant

- Toda tabela operacional tem `tenant_id`
- RLS: `is_platform_super_admin()` OR `tenant_id IN get_my_tenant_ids()`
- Trigger `set_default_tenant_id()` no INSERT
- Tenant ativo em `localStorage` (`sb-autoclaims-tenant-id`)

4. Estrutura de pastas

```
??? api/                # Serverless Vercel
?   ??? _lib/           # auth, supabase admin, http helpers
?   ??? v1/            # API REST pública (API Key)
?   ??? cnpj/          # Proxy BrasilAPI
?   ??? vehicles/      # Lookup placa
?   ??? llm/           # Proxy Gemini
??? components/         # UI reutilizável
??? context/           # AuthContext, ToastContext
??? hooks/             # useEventTypes, etc.
??? pages/             # Uma página por módulo
??? services/          # supabaseClient, Service.ts
??? supabase/migrations/ # SQL versionado
??? utils/             # impressões, defaults, matrixPrint
??? docs/              # Documentação técnica
??? .github/workflows/ # CI/CD
```

5. Módulos funcionais (pages/)

```
Rota | Arquivo | Responsabilidade |
-----|-----|-----|
/ | Dashboard.tsx | KPIs, gráficos, atalhos |
/eventos | Events.tsx | Sinistros: veículo associado/veículo, anexos, cota participativa |
```

/cotacoes | Quotations.tsx | RFQ wizard, matriz, cadastro r?pido sinistro |
 /compras | Purchases.tsx | OCs agrupadas por caso, aprova??o escrita, auditoria |
 /entregas | Deliveries.tsx | Fila entregas, marcar entregue, editar/excluir |
 /fornecedores | Suppliers.tsx | Parceiros, CNPJ/CEP auto, avalia??es |
 /associados | Associates.tsx | Clientes PF/PJ, CNPJ lookup |
 /veiculos | Vehicles.tsx | Frota, lookup placa |
 /catalogo | Catalog.tsx | Pe?as e servi?os padronizados |
 /relatorios | Reports.tsx | BI (requer view_reports) |
 /configuracoes | Settings.tsx | IA, equipe, API keys, auditoria |
 /saas-admin | SaasAdmin.tsx | Super admin tenants/planos |
 /login | Login.tsx | Auth email/OAuth |
 /register | Register.tsx | Nova empresa ou convite |
 /auth/callback | AuthCallback.tsx | OAuth/email confirm |

6. Fluxos de neg?cio cr?ticos

6.1 Sinistro (Events)

1. Selecionar Associado / Terceiro e Ve?culo (ou cadastro r?pido inline)
2. Tipo sinistro (configur?vel em Settings -> Tipos, inclui Acordo)
3. Cota de participa??o do ve?culo (campo num?rico opcional)
4. Anexos multim?dia -> Supabase Storage bucket event-attachments
5. Viewer in-browser (FileViewerModal) para imagem/v?deo/PDF
6. Exclus?o via RPC delete_event_cascade (valida retorno)

6.2 Cota??o (Quotations)

1. Wizard: selecionar sinistro OU cadastro r?pido (nome, placa, CNPJ, cota, anexos)
2. Itens do cat?logo (pe?as/servi?os)
3. Convidar fornecedores
4. Matriz (MatrixTable): pre?os manuais/simulados, sele??o manual (n?o auto)
5. Impress?o matriz: preview Paisagem ou Lista (utils/matrixPrint.ts)

6.3 Compra (Purchases)

1. quotationService.processPurchase() gera 1 OC por fornecedor (status Gerada)
2. Aprova??o exige justificativa por escrito + permiss?o approve_purchases
3. Trigger DB valida permiss?o e nota obrigat?ria
4. Hist?rico auditoria (purchase_order_history) autom?tico + UI timeline
5. Preview impress?o OC antes de print (purchaseOrderPrint.ts)

6.4 Entregas (Deliveries)

1. OCs aprovadas aparecem na fila
2. Marcar entregue: respons?vel + observa??o -> status OC Recebida
3. Editar/excluir entregas (ActionModal estilizado)
4. Hist?rico movimenta??es em deliveries.movement_history (JSONB)

7. Autentica??o e seguran?a

Login

- Supabase Auth PKCE + storage h?brido (authStorage.ts)
- Google OAuth -> /auth/callback
- Sem membership -> logout for?ado (sem "modo r?pido")
- PrivateRoute: exige user + membership ou super_admin

Permiss?es (services/accessControl.ts)

Flag | Efeito |

```

-----|-----|
approve_purchases | Aprovar/cancelar OC |
delete_records    | Excluir registros |
financial_view    | Valores financeiros |
view_reports     | /relatorios |
manage_users     | Equipe |

```

Gerente = Admin/Gerente profile OU owner/admin membership.

Rotas guardadas

- PermissionRoute em /configuracoes, /relatorios
- AdminRoute em /saas-admin

RPCs sensíveis

- complete_registration, accept_invite, get_tenant_members
- update_tenant_member_profile, detach_tenant_member
- create_tenant_api_key, revoke_tenant_api_key, validate_api_key
- delete_event_cascade, delete_tenant_cascade

Documentação: docs/SEGURANCA-E-PERMISSOES.md

8. Banco de dados (principais tabelas)

```

Tabela | Propósito |
-----|-----|
saas_tenants | Empresas clientes |
organization_members | Usuário ? tenant |
profiles | Perfil, role, permissions JSONB |
invitations | Convites pendentes |
events | Sinistros |
event_history | Histórico status sinistro |
event_attachments | Anexos sinistro |
associates | Associados/terceiros |
vehicles | Veículos |
suppliers | Fornecedores |
catalog_items | Catálogo peças/serviços |
quotations | Cotações RFQ |
quotation_items | Itens da cotação |
quotation_supplier_prices | Preços matriz |
quotation_purchase_selections | Seleção manual compra |
purchase_orders | Ordens de compra |
purchase_order_items | Itens OC (normalizado) |
purchase_order_history | Auditoria OC |
deliveries | Entregas |
audit_logs | Auditoria global |
api_keys | Chaves integramento REST |
saas_settings | Config tenant (IA, tipos sinistro) |

```

Campos recentes:

- events.participation_quota, quotations.participation_quota, quotations.attachments
- purchase_orders.approval_note, approved_by, approved_at

9. API REST v1 (integração externa)

Base URL: <https://eventos.escsistemas.com/api/v1>

Autenticação: Header Authorization: Bearer evsc_live_... ou X-API-Key

Gerar chave: Configurações -> API / Integrações

Método	Endpoint	Auth	Descrição
GET	/health	Não	Health check
GET	/	Não	Metadados endpoints
GET	/events	Sim	Listar sinistros
GET	/quotations	Sim	Listar cotações (id= detalhe)
GET	/purchase-orders	Sim	Listar OCs (id= com history)
GET	/deliveries	Sim	Listar entregas
GET	/associates	Sim	Listar associados

Implementação: api/v1/.js + api/_lib/auth.js (valida hash via RPC validate_api_key)

Documentação completa: docs/API-INTEGRACAO.md

APIs auxiliares (sem API Key)

Endpoint	Uso
GET /api/cnpj/lookup?cnpj=	Razão social, CEP, endereço
GET /api/vehicles/lookup?plate=	Dados veículo
POST /api/llm/generate	Proxy IA (Gemini)

10. Serviços frontend (services/)

Arquivo	Função
supabaseClient.ts	Cliente Supabase PKCE
eventService.ts	CRUD sinistros, anexos, delete cascade
quotationService.ts	Matriz, processPurchase, simulação
purchaseOrderService.ts	Histórico auditoria OC
attachmentService.ts	Upload/viewer anexos Storage
lookupService.ts	CNPJ, CEP, placa
quickRegisterService.ts	Cadastro rápido associado/veículo
accessControl.ts	Resolve permissões UI
auditService.ts	Logs audit_logs
apiKeyService.ts	CRUD chaves API

11. Componentes UI importantes

Componente	Uso
PremiumModal	Modais formulário (sinistro)
ActionModal	Confirmações estilizadas (substituir window.confirm)
PermissionRoute	Guard rotas por permissão
PrivateRoute	Guard autenticação
MatrixTable	Matriz cotação comparativa
FileViewerModal	Visualizar vídeo/imagem/PDF
Layout	Sidebar navegação

Padrão visual: rounded-2xl/3xl, font-black uppercase tracking-widest labels, blue-600 primary, cards brancos border slate-200.

12. Realtime

Subscriptions Supabase em:

- Purchases.tsx -> purchase_orders, quotations
- Deliveries.tsx -> deliveries, purchase_orders

13. CI/CD

.github/workflows/deploy.yml:

1. build - npm ci && npm run build
2. migrate (main) - supabase db push
3. deploy-production - Vercel com retry + inspect Ready
4. deploy-preview - PRs

Autor git deploy: 257080210+devgpesc@users.noreply.github.com

14. Migrations essenciais (ordem lógica)

1. 20240401_saas_architecture_v2.sql - multi-tenant
2. 20260525114135_enforce_operational_tenant_rls.sql - RLS tenant
3. 20260524232239_harden_tenant_user_permissions.sql - RPCs equipe
4. 20260716220000_sinistro_entregas_anexos.sql - anexos, approval OC
5. 20260716230000_delete_event_cascade_rpc.sql - delete sinistro
6. 20260717010000_participation_quota_quotation_attachments.sql - cota RFQ
7. 20260717030000_po_history_api_keys_security.sql - auditoria OC, API keys, guards

15. Regras de implementação (para o Agent Cursor)

1. Responda em português na UI e toasts
2. Nunca usar window.confirm - usar ActionModal
3. Sempre verificar { error } em mutations Supabase
4. Respeitar access. antes de ações sensíveis
5. Manter isolamento tenant - nunca query sem RLS
6. Cadastro reutilizar quickRegisterService
7. Anexos reutilizar attachmentService + ATTACHMENT_ACCEPT
8. Impressão abrir preview no navegador antes de window.print()
9. Commits só quando usuário pedir; mensagens em português
10. Build deve passar: npm run build

16. Checklist funcional (aceite)

- [] Login/logout/convite/registro empresa
- [] Sinistro com associado/terceiro, veículo, cota, anexos veículo
- [] Cota RFQ com anexos e cadastro reutilizar
- [] Matriz impressão paisagem/lista
- [] OC aprova por escrito + auditoria timeline
- [] Entregas editar/excluir/marcar entrega
- [] Fornecedor CNPJ+CEP automático
- [] Catálogo CRUD com ActionModal delete
- [] API REST v1 com chaves em Settings
- [] Permissões bloqueiam rotas e ações
- [] Deploy CI verde

17. Prompt de execução (copiar e colar)

Você é o agente principal do EventsCar (GP-DEV-SINISTRO).

Stack: React + TypeScript + Vite + Supabase + Vercel serverless.

Multi-tenant com RLS. Produto: eventos.escsistemas.com.

Implemente/mantenha TODOS os módulos descritos neste documento:

- Sinistros, Cotas, Compras (OC+auditoria), Entregas, Fornecedores, Associados, Veículos, Catálogo, Relatórios, Configurações, SaaS Admin.

Requisitos obrigatórios:

1. Segurança: PermissionRoute, accessControl, triggers DB em OCs
2. API REST v1 com API Keys (events, quotations, purchase-orders, deliveries, associates)
3. ActionModal em exclusões; sem window.confirm
4. Anexos multimídia com FileViewerModal
5. Impressão sempre com preview no browser
6. Migrations Supabase versionadas em supabase/migrations/
7. npm run build sem erros antes de commit

Ao alterar schema, crie migration SQL.

Ao criar endpoints, atualize docs/API-INTEGRACAO.md.

Documente decisões em docs/ quando relevante.

Comece lendo: App.tsx, AuthContext, accessControl.ts, supabase/migrations mais recentes.

18. Referências de arquivos-chave

pages/Events.tsx
pages/Quotations.tsx
pages/Purchases.tsx
pages/Deliveries.tsx
pages/Settings.tsx
components/MatrixTable.tsx
components/ActionModal.tsx
services/quotationService.ts
services/eventService.ts
services/purchaseOrderService.ts
utils/matrixPrint.ts
utils/purchaseOrderPrint.ts
api/v1/
supabase/migrations/
.github/workflows/deploy.yml
docs/API-INTEGRACAO.md
docs/SEGURANCA-E-PERMISSOES.md
docs/PROMPT-ANEXOS-VIDEO-CURSOR.md

Documento gerado para replicação e manutenção do EventsCar via Cursor IDE.

Versão: 2026-07-17

API REST - Integração

EventsCar - Documentação de Integração via API (v1)

Guia técnico para conectar sistemas externos ao EventsCar (GP-DEV-SINISTRO).

Visão geral

Item	Valor
Versão	v1
Base URL (produto)	https://eventos.escsistemas.com/api/v1
Autenticação	API Key (Bearer ou header)
Formato	JSON
Charset	UTF-8

Autenticação

Gerar chave

1. Acesse Configurações -> API / Integrações (perfil Admin/Gerente).
2. Informe um nome descritivo (ex: ERP Matriz).
3. Clique em Gerar chave.
4. Copie a chave imediatamente - ela não é exibida novamente.

Formato da chave:

```
text
evsc_live_<64 caracteres hexadecimais>
```

Enviar a chave nas requisições

Opção 1 - Authorization Bearer (recomendado)

```
http
GET /api/v1/events HTTP/1.1
Host: eventos.escsistemas.com
Authorization: Bearer evsc_live_...
Accept: application/json
```

Opção 2 - Header dedicado

```
http
X-API-Key: evsc_live_...
```

Escopos

Escopo	Descrição
read	Leitura de sinistros e ordens de compra
admin	Reservado para expansões futuras

Chaves revogadas retornam 401.

Endpoints

Health check (público)

```
http
GET /api/v1/health
```

Resposta 200

```
json
{
  "ok": true,
  "service": "EventsCar API",
  "version": "v1",
  "timestamp": "2026-07-17T03:00:00.000Z"
}
```

Metadados da API

```
http
GET /api/v1
```

Lista endpoints disponíveis e formas de autenticação.

Sinistros (Events)

```
http
GET /api/v1/events
Authorization: Bearer evsc_live_...
```

Query params

Param	Tipo	Descrição
limit	number	Máx. 200 (padrão 50)
status	string	Filtrar por status (Aguardando, Em Cotação, etc.)

Resposta 200

```
json
{
  "data": [
    {
      "id": "uuid",
      "protocol": "EVT-2026-0004",
      "type": "Colisão",
      "category": "Colisão",
      "status": "Aprovado",
      "priority": "Média",
      "associateId": "uuid",
      "vehicleId": "uuid",
      "participation_quota": 200,
      "created_at": "2026-07-16T12:00:00.000Z",
      "tenant_id": "uuid"
    }
  ]
}
```

```

    }
  ],
  "meta": {
    "count": 1,
    "tenant_id": "uuid"
  }
}

```

Cotações (Quotations)

```

http
GET /api/v1/quotations
Authorization: Bearer evsc_live_...

```

Query params: limit, code, status, id (detalhe com itens)

Entregas (Deliveries)

```

http
GET /api/v1/deliveries
Authorization: Bearer evsc_live_...

```

Query params: limit, status (Pendente, Entregue), po (código OC)

Associados (Associates)

```

http
GET /api/v1/associates
Authorization: Bearer evsc_live_...

```

Query params: limit, document (CPF/CNPJ sem máscara)

Ordens de compra (Purchase Orders)

Listar OCs

```

http
GET /api/v1/purchase-orders
Authorization: Bearer evsc_live_...

```

Query params

Param	Tipo	Descrição
limit	number	Máx. 200 (padrão 50)
code	string	Filtra por código (OC-2026-12345)
status	string	Gerada, Aprovada, Recebida, Cancelada

Resposta 200

```

json
{
  "data": [
    {
      "id": "uuid",
      "code": "OC-2026-58658",
      "status": "Aprovada",
      "total": 400,
      "created_at": "2026-07-16T12:00:00.000Z",
      "approval_note": "Aprovado conforme cota??o vencedora.",
      "approved_at": "2026-07-16T14:00:00.000Z",
      "quotation_id": "uuid",
      "event_id": "uuid",
      "supplier_id": "uuid",
      "suppliers": { "name": "Fornecedor XYZ", "city": "Goi?nia" },
      "purchase_order_items": [
        {
          "name": "Lanterna traseira LD",
          "quantity": 1,
          "unit": "UN",
          "unit_price": 200,
          "total_price": 200
        }
      ]
    }
  ],
  "meta": { "count": 1, "tenant_id": "uuid" }
}

```

Detalhe com hist?rico de auditoria

```

http
GET /api/v1/purchase-orders?id=<uuid-da-oc>
Authorization: Bearer evsc_live...

```

Resposta 200 - inclui array history:

```

json
{
  "data": {
    "id": "uuid",
    "code": "OC-2026-58658",
    "status": "Aprovada",
    "history": [
      {
        "id": "uuid",
        "action": "created",
        "from_status": null,
        "to_status": "Gerada",
        "comment": null,
        "user_id": "uuid",
        "created_at": "2026-07-16T12:00:00.000Z"
      },
      {

```

```

    "id": "uuid",
    "action": "approved",
    "from_status": "Gerada",
    "to_status": "Aprovada",
    "comment": "Aprovado por escrito.",
    "created_at": "2026-07-16T14:00:00.000Z"
  }
]
}
}

```

Ações de auditoria

action	Descrição
created	OC gerada a partir da cotação
approved	Aprovação por escrito
cancelled	OC cancelada
received	Recebida / entregue
updated	Outras alterações
deleted	Registro excluído

Códigos HTTP

Código	Significado
200	Sucesso
204	OPTIONS (CORS preflight)
400	Parâmetro inválido
401	Chave ausente, inválida ou revogada
405	Método não permitido
500	Erro interno

Exemplos

cURL - listar sinistros

```

bash
curl -s "https://eventos.escsistemas.com/api/v1/events?limit=10" \
  -H "Authorization: Bearer evsc_live_SUA_CHAVE_AQUI" \
  -H "Accept: application/json"

```

cURL - OC com auditoria

```

bash
curl -s "https://eventos.escsistemas.com/api/v1/purchase-orders?id=UUID_DA_OC" \
  -H "Authorization: Bearer evsc_live_SUA_CHAVE_AQUI"

```

JavaScript (fetch)

```

javascript
const API_KEY = process.env.EVENTSCAR_API_KEY;
const BASE = 'https://eventos.escsistemas.com/api/v1';

```

```

async function listPurchaseOrders() {
  const res = await fetch(`${BASE}/purchase-orders?status=Aprovada, {
    headers: {
      Authorization: Bearer ${API_KEY},
      Accept: 'application/json',
    },
  });
  if (!res.ok) throw new Error(await res.text());
  return res.json();
}

```

Python (requests)

```

python
import os
import requests

API_KEY = os.environ["EVENTSCAR_API_KEY"]
BASE = "https://eventos.escsistemas.com/api/v1"

response = requests.get(
    f"{BASE}/events",
    headers={"Authorization": f"Bearer {API_KEY}"},
    params={"limit": 20},
    timeout=30,
)
response.raise_for_status()
print(response.json())

```

Segurança e permissões (aplicação web)

Modelo de acesso

Camada	Descrição
Tenant	Cada empresa (saas_tenants) isola dados via RLS
Membership	organization_members.role: owner, admin, member, observer
Profile	profiles.role: Admin, Gerente, Usuário, super_admin
Permissões granulares	JSON em profiles.permissions

Permissões disponíveis

Chave	Efeito
approve_purchases	Aprovar e cancelar OCs
delete_records	Excluir OCs e registros
financial_view	Ver valores financeiros detalhados
view_reports	Acessar relatórios BI
manage_users	Gerenciar equipe e convites

Administradores da empresa (Admin/Gerente ou owner/admin) possuem todas as permissões operacionais.

Regras enforced no banco

- Aprovação de OC exige approval_note e permissão approve_purchases (trigger PostgreSQL).
- Exclusão de OC exige permissão delete_records ou perfil gerencial.

- Itens de OC (purchase_order_items) isolados por tenant via ordem pai.
- API Keys armazenadas apenas como hash SHA-256.

Variáveis de ambiente (servidor Vercel)

Variável	Obrigatória	Uso
SUPABASE_URL	Sim	URL do projeto Supabase
SUPABASE_SERVICE_ROLE_KEY	Sim	Validação de API keys e queries server-side
VITE_SUPABASE_URL	Fallback	Usado se SUPABASE_URL não estiver definida

Limitações v1

- Apenas leitura (GET) via API pública.
- Escrita (criar sinistro, aprovar OC) permanece no app autenticado ou via Supabase direto com JWT do usuário.
- Rate limiting: aplicar no gateway (Vercel/WAF) conforme volume.

Suporte

- Repositório: devgpesc/GP-DEV-SINISTRO
- Migration relacionada:
supabase/migrations/20260717030000_po_history_api_keys_security.sql

Segurança, Login e Permissões

EventsCar - Segurança, Login e Permissões

Fluxo de autenticação

1. Login (/login) - e-mail/senha ou Google OAuth.
2. Registro (/register) - empresa nova (complete_registration) ou convite (accept_invite).
3. Callback (/auth/callback) - confirma e-mail, OAuth PKCE, finaliza convite.
4. Sessão - Supabase Auth com PKCE; tenant ativo em localStorage (sb-autoclaims-tenant-id).

Usuário sem membership em nenhuma empresa é deslogado automaticamente (exceto super_admin ou fluxo de convite).

Camadas de permissões

Nível	Onde	Valores
Papel global	profiles.role	super_admin, Admin, Gerente, Usuário
Papel na empresa	organization_members.role	owner, admin, member, observer
Flags	profiles.permissions (JSON)	Ver tabela abaixo

Flags granulares

Flag	Uso no sistema
approve_purchases	Aprovar e cancelar ordens de compra
delete_records	Excluir OCs e registros permanentes
financial_view	Valores financeiros detalhados
view_reports	Rota /relatorios
manage_users	Equipe, convites, aba Usuários

Gerentes (Admin/Gerente) e owner/admin da empresa herdam permissões operacionais completas.

Rotas protegidas

Rota	Requisito
/* (app)	Sessão válida + membership ou super_admin
/configuracoes	canManageSettings
/relatorios	canViewReports
/saas-admin	super_admin

Ordens de compra - regras no banco

Triggers PostgreSQL (guard_purchase_order_mutations):

- Aprovar -> exige approve_purchases + approval_note preenchido.
- Cancelar -> exige approve_purchases.
- Excluir -> exige delete_records ou perfil gerencial.

Histórico automático em purchase_order_history (trigger log_purchase_order_history).

Gestão de equipe

- Listar/editar: RPC get_tenant_members, update_tenant_member_profile.
- Remover da empresa: detach_tenant_member (não apaga conta auth).
- Convites: tabela invitations com token; aceite valida e-mail.

Boas práticas

- Não compartilhar chaves de API - revogar em Configurações -> API / Integrações.
- Conceder delete_records apenas a perfis de confiança.
- Revisar aba Auditoria periodicamente (audit_logs).

Anexos de Video (modulo)

Prompt Cursor - Módulo de Anexos com Vídeo (Supabase Storage + React)

Use este prompt na sua IDE Cursor para implementar em outro sistema a mesma funcionalidade de anexos multimídia com visualização no navegador.

Prompt

Implemente um módulo completo de anexos de evidências em um sistema React + Supabase com os requisitos abaixo.

Stack

- React + TypeScript + Vite
- Supabase (PostgreSQL + Storage)
- Tailwind CSS (opcional, mas recomendado)

Objetivo

Permitir upload, listagem, remoção e visualização no navegador de arquivos anexados a um registro pai (ex: sinistro, chamado, processo).

Formatos suportados

- Imagens: jpg, png, gif, webp
- Vídeos: mp4, mov, webm, avi, mkv
- Documentos: pdf, doc, docx, xls, xlsx, txt, zip, rar

Banco de dados

Crie tabela event_attachments (ou equivalente genérica):

- id uuid PK
- parent_id uuid FK (ex: event_id)
- url text NOT NULL
- name text
- type text
- created_at timestampz default now()

Supabase Storage

- Bucket público: event-attachments (ou nome genérico)
- Path: {parent_id}/{timestamp}-{filename}
- Políticas: authenticated INSERT/UPDATE/DELETE; public SELECT

Serviço attachmentService.ts

Funções:

1. getAttachmentKind(mime, filename) -> 'image' | 'video' | 'pdf' | 'word' | 'file'
2. uploadAttachments(parentId, attachments[]) -> salva no Storage + insere linhas na tabela
3. deleteAttachment(id, url?) -> remove registro + arquivo do Storage

Interface local antes do upload:

ts

```
interface Attachment {
  id?: string;
  name: string;
  type: string;
  url: string;
  size?: string;
  file?: File;
  isNew?: boolean;
```

```
}
```

Componente `FileViewerModal.tsx`

Modal fullscreen/lightbox que renderiza:

- image -> tag `<img>`
- video -> tag `<video controls autoPlay>`
- pdf -> `<iframe src={url}>`
- word/outros -> mensagem + botões Abrir/Baixar

Incluir botões: Baixar, Abrir em nova aba, Fechar.

Formulário de upload

- Input file: `accept="image/,video/,application/pdf,.pdf,.doc,.docx,.xls,.xlsx,.txt,.zip,.rar,.mp4,.mov,.avi,.webm,.mkv"`
- Preview por tipo (thumbnail para imagem, ícone para vídeo/doc)
- Botão "Abrir" em cada anexo
- Botão remover

Fluxo de persistência

1. Usuário seleciona arquivos -> ficam em memória com `URL.createObjectURL` para preview
2. Ao salvar o registro pai -> chama `uploadAttachments(parentId, newOnes)`
3. Ao editar -> carregar anexos existentes via `join/select`
4. Remover anexo existente -> `deleteAttachment`

UX

- Não usar `window.open` direto para impressão; preferir modal interno
- Tratar erro de upload com toast/mensagem clara
- Limitar tamanho recomendado: aviso acima de 50MB

Entregáveis

1. Migration SQL (tabela + bucket + policies)
2. `services/attachmentService.ts`
3. `components/FileViewerModal.tsx`
4. Integração em formulário de exemplo
5. README curto de configuração Supabase

Siga boas práticas: tipagem forte, sem expor service role no frontend, URLs públicas do bucket ou signed URLs se preferir bucket privado.

Referência de implementação

No projeto GP-DEV-SINISTRO, os arquivos de referência são:

- `services/attachmentService.ts`
- `components/FileViewerModal.tsx`
- `utils/defaults.ts` (constante `ATTACHMENT_ACCEPT`)
- `supabase/migrations/20260716220000_sinistro_entregas_anexos.sql`